CS619 Final Project Final Viva Preparation

Scope of the project

Project scope is the part of project planning that involves determining and documenting a list of specific project goals, deliverables, features, functions, tasks, deadlines, and ultimately costs. In other words, it is what needs to be achieved and the work that must be done to deliver a project.

Functional Requirements Non Functional requirements

Functional Requirements

A functional requirement describes what a software system should do,

Functional Requirements: specify the functionality of the system.

Functional requirements are the activities that a system must perform

Examples

Add customer

Print invoice

Display the name

Non Functional Requirements

Non-functional requirements describe how the system works,

how the system should behave and that it is a constraint upon the systems behavior.

Examples

Performance:

Application should perform all of its functions excellently and effectively without any inconvenience.

Availability:

This application will be available free for everyone

Portability:

The application can easily portable from one desktop application to another

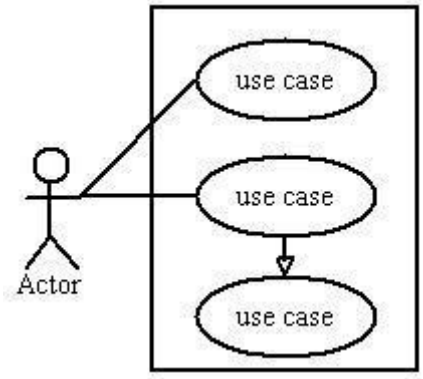
User Friendly:

Application will have a user friendly interface.

Use Case Diagram

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved.

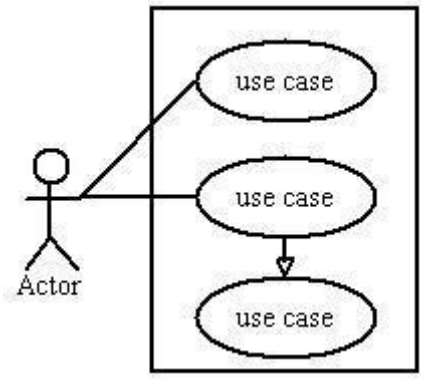
Basic Use Case Diagram Symbols and Notations



System

Draw your system's boundaries using a rectangle that contains use cases. Place actors outside the system's boundaries.

System is used to define the scope of the use case and drawn as a rectangle. This an optional element but useful when your visualizing large systems. For example you can create all the use cases and then use the system object to define the scope covered by your project. Or you can even use it to show the different areas covered in different releases.



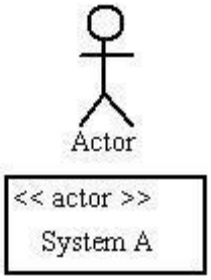
Use Case

Draw use cases using ovals. Label with ovals with verbs that represent the system's functions.



Actors

Actors are the users of a system. When one system is the actor of another system, label the actor system with the actor stereotype.



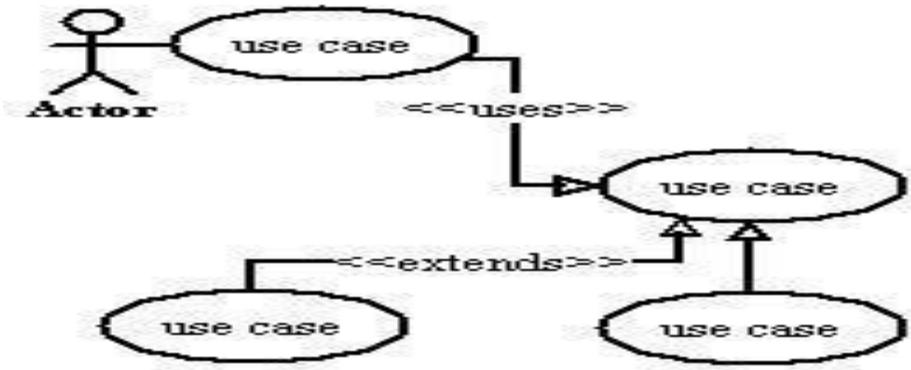
Relationships

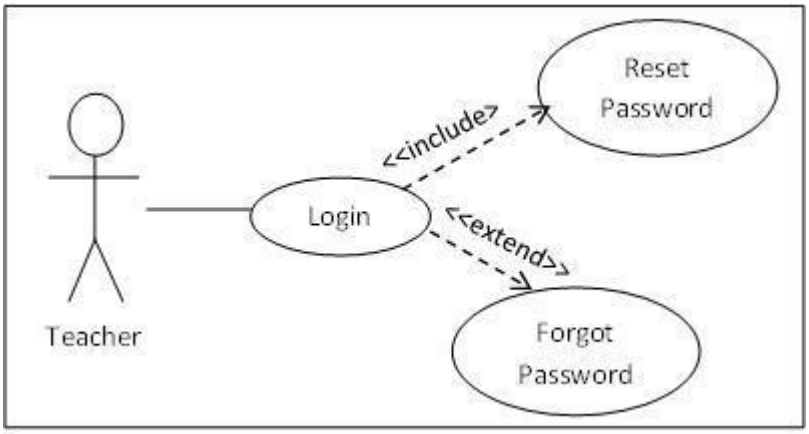
Illustrate relationships between an actor and a use case with a simple line. For relationships among use cases, use arrows labeled either "uses" or "extends" or "includes

A "uses" relationship indicates that one use case is needed by another in order to perform a task.

An "extends" relationship indicates alternative options under a certain use case.

An include relationship is a relationship between two use cases It indicates that the use case to which the arrow points is included in the use case on the other side of the arrow. This makes it possible to reuse a use case in another use case.





The following is a sample use case diagram representing the order management system. So if we look into the diagram then we will find three use cases (Order, Special Order and Normal Order) and one actor which is customer.

The Special Order and Normal Order use cases are extended from Order use case. So they have extends relationship. Another important point is to identify the system boundary which is shown in the picture. The actor Customer lies outside the system as it is an external user of the system.

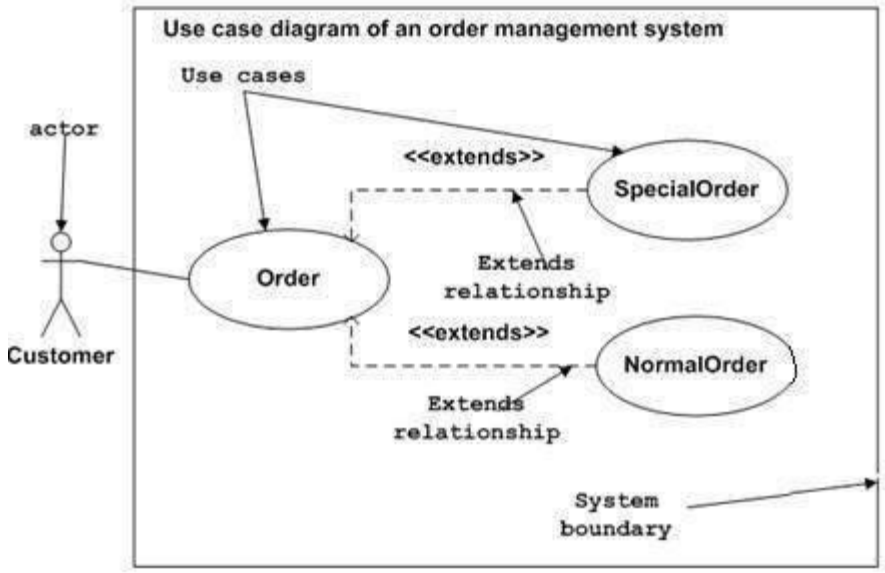


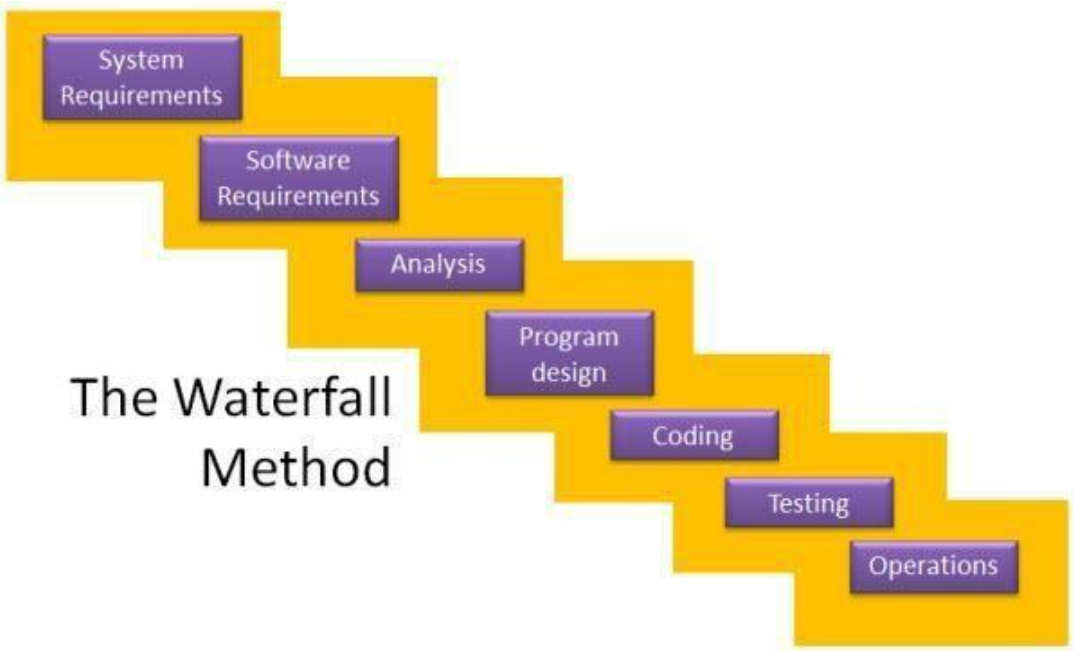
Figure: Sample Use Case diagram

Adopted Methodology

It is a method to use development of project. We have adopted the VU Process Model which is the combination of Waterfall methodology and Spiral methodology. VU Process Model will combine the benefits of Waterfall and Spiral methodologies.

The Waterfall Model

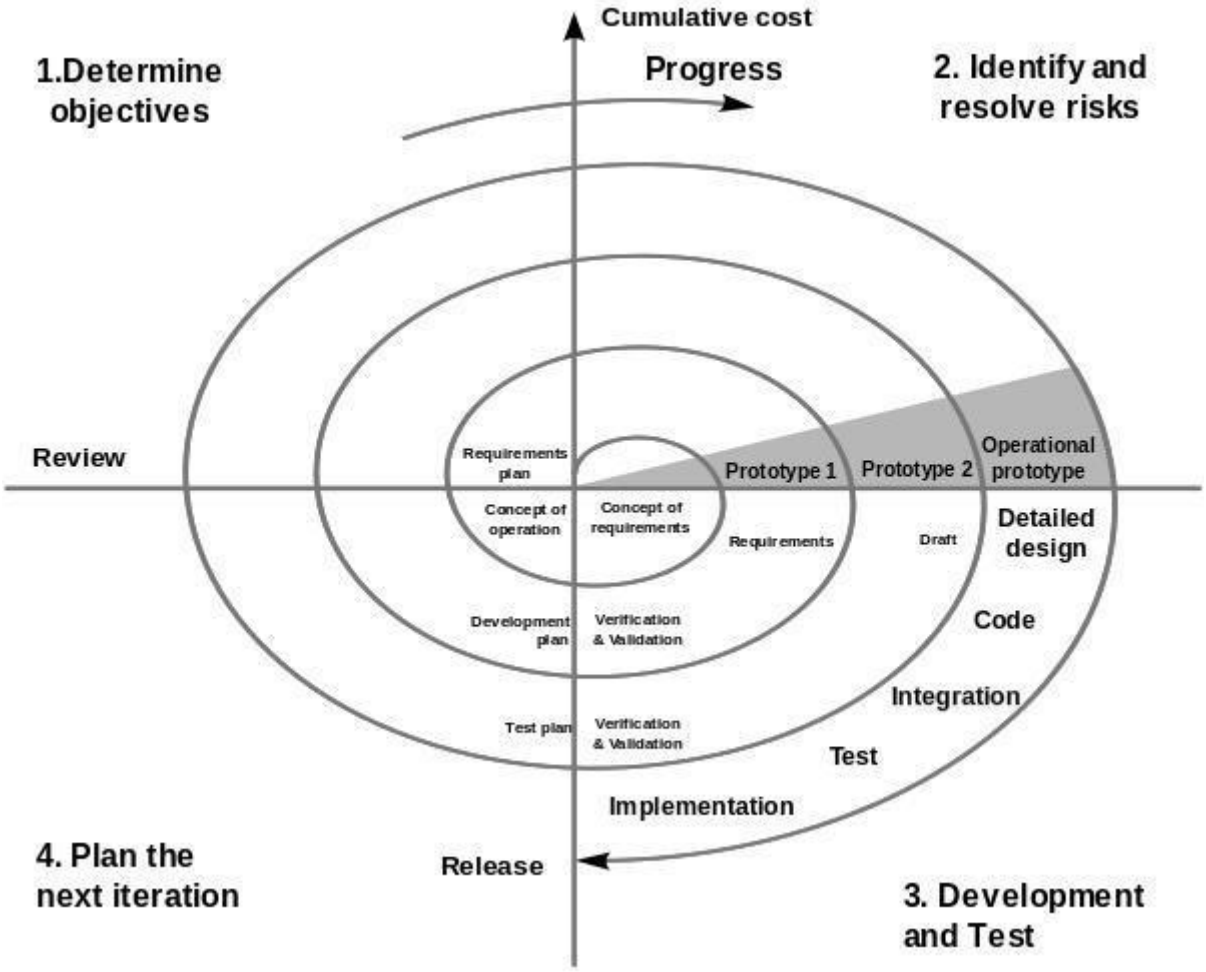
The waterfall model is a sequential design process, used in software development processes, in which progress is seen as flowing steadily downwards (like a waterfall) through the phases of Conception, Initiation, Analysis, Design, Construction, Testing, Production/Implementation and Maintenance.



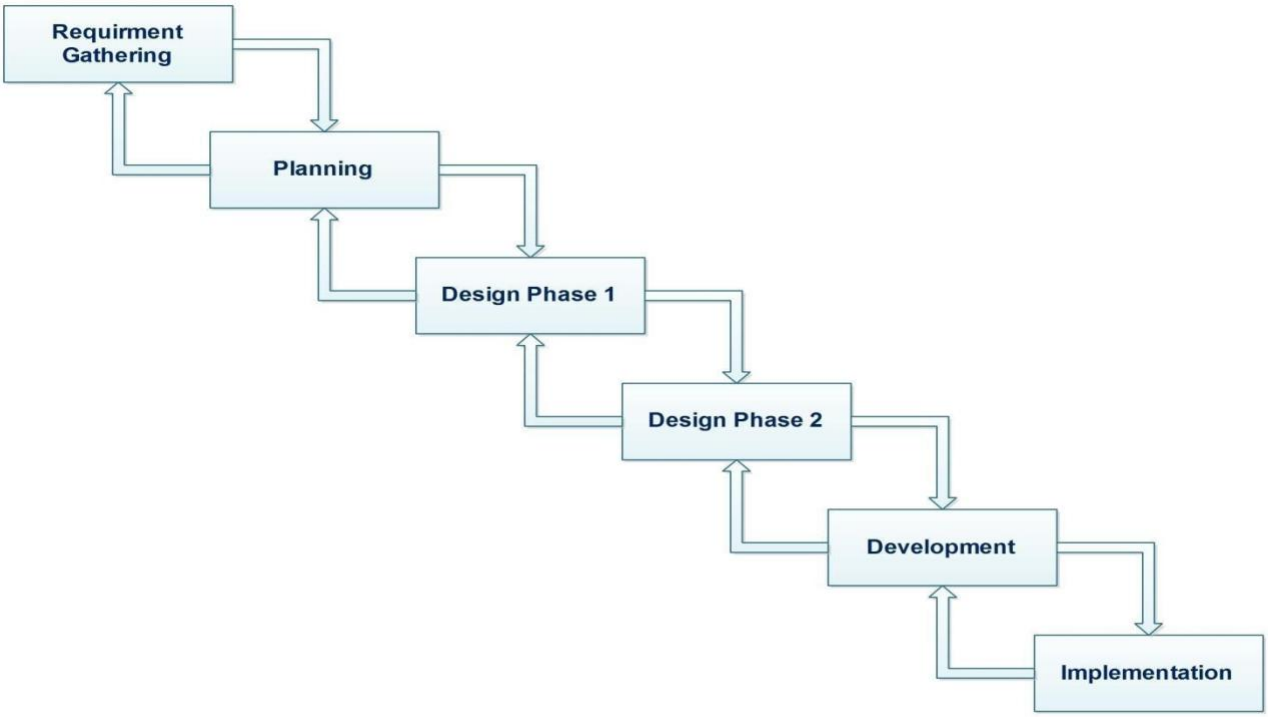
The spiral model

The spiral model, also known as the spiral lifecycle model, is a systems development method (SDM) used in information technology (IT). This model of development combines the features of the prototyping model and the systems development life cycle (SDLC).

VU



Process Model



It is decided to select VU process Model that is a Combination of Water Fall model and Spiral Model.

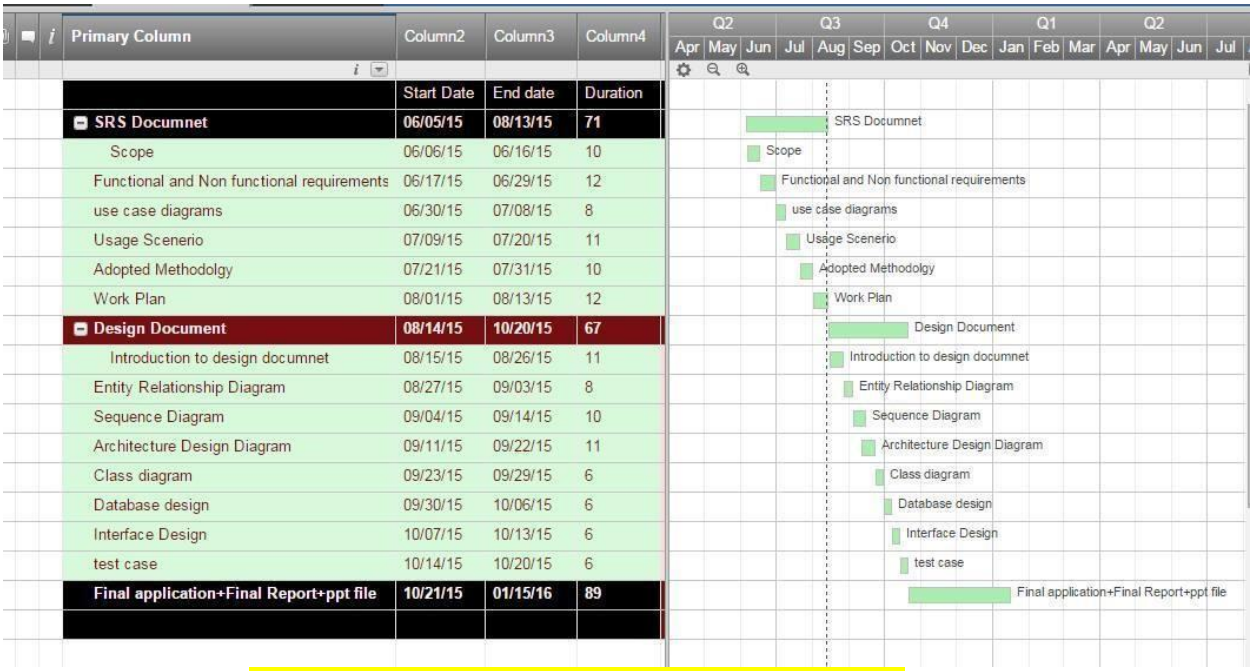
Reasons for Choosing VU process Model.

1. The main idea to choose is to get the benefits of both these models.
2. It is sequential model with backward repetition.
3. We want to achieve the linear nature of waterfall and iterative plus risk reduction nature of spiral model.
4. In vu process model we will be work in phases to complete the given project.
5. All the activities are performed in a sequence in VU Process Model.

If we want to do correction or betterment at any stage and repetition whenever necessary then we can choose VU process model

Work Plan / Gantt Chart

The work plan outlines the plan by which the company plans to complete a quality project within a given amount of time and in compliance with a set budget.



Primary Column	Column2	Column3	Column4
	Start Date	End date	Duration
SRS Documnet	**06/05/15**	**08/13/15**	**71**
Scope	06/06/15	06/16/15	10
Functional and Non functional requirements	06/17/15	06/29/15	12
use case diagrams	06/30/15	07/08/15	8
Usage Scenerio	07/09/15	07/20/15	11
Adopted Methodolgy	07/21/15	07/31/15	10
Work Plan	08/01/15	08/13/15	12
Design Document	**08/14/15**	**10/20/15**	**67**
Introduction to design documnet	08/15/15	08/26/15	11
Entity Relationship Diagram	08/27/15	09/03/15	8
Sequence Diagram	09/04/15	09/14/15	10
Architecture Design Diagram	09/11/15	09/22/15	11
Class diagram	09/23/15	09/29/15	6
Database design	09/30/15	10/06/15	6
Interface Design	10/07/15	10/13/15	6
test case	10/14/15	10/20/15	6
Final application+Final Report+ppt file	**10/21/15**	**01/15/16**	**89**

Entity Relationship Diagram (ERD)

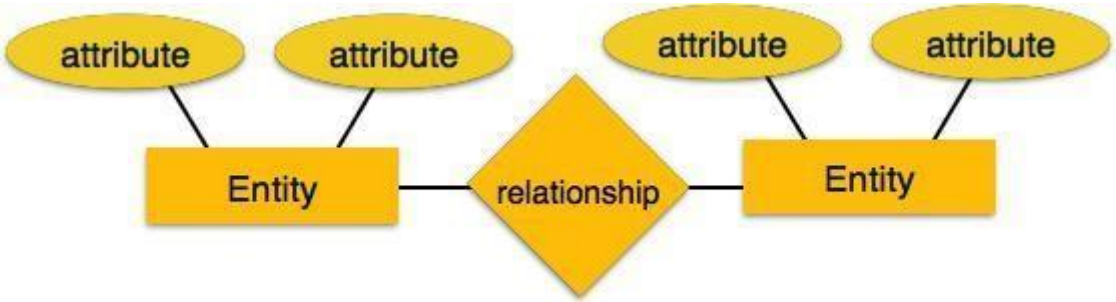
An entity-relationship diagram—otherwise known as an ERD—is a data modeling technique that creates an illustration of an information system's entities and the relationships between those entities.

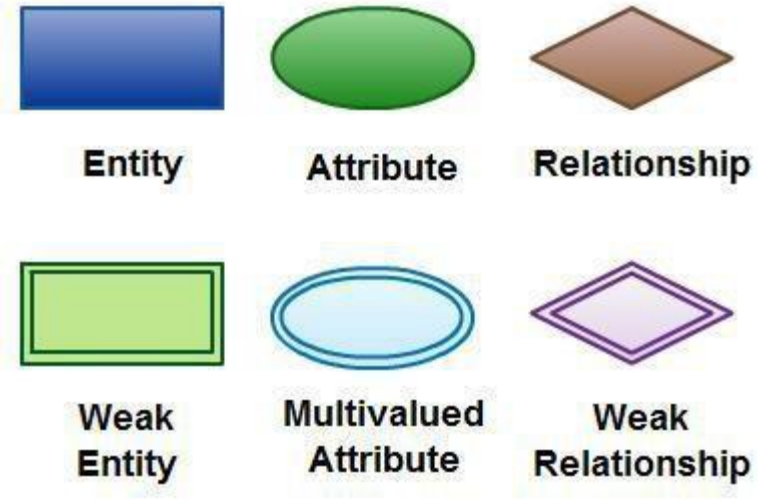
3 ingredients of entity-relationship diagram

Entities, which represent people, places, items, events, or concepts.

Attributes, which represent properties or descriptive qualities of an entity. These are also known as data elements.

Relationships, which represent the link between different entities.





ER Diagram Symbols and Notations

Cardinality and Modality

Cardinality and Modality work together to define the relationship.

- Cardinality indicates the maximum number of times an instance in one entity can be associated with instances in the related entity.□

- Modality indicates the minimum number of times an instance in one entity can be associated with an instance in the related entity.□

Thus, Modality is also called participation because it denotes whether or not an instance of an entity MUST participate in the relationship.

Cardinality and Modality are both shown on the relationship line by symbols. We will go over each of the symbols and how to interpret them.

Cardinality

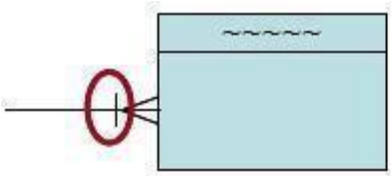


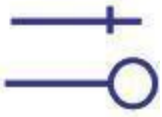
Cardinality indicates the maximum number of times an instance of one entity can be associated with instances in the related entity. Cardinality can have the values of one or many, no more detail than that. It is either one or more than one. On the relationship line,

the cardinality is the closest to the entity box. The cardinality symbol in the diagram on the slide is in the red circle. Cardinality is indicated at BOTH ends of the relationship line, so there is a left to right cardinality and a right to left cardinality.

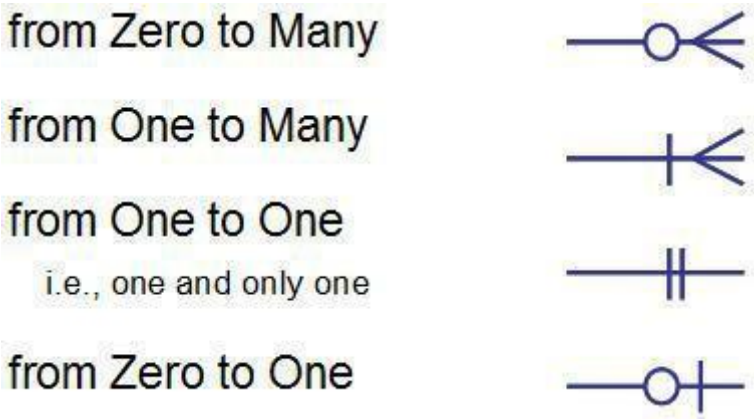
Modality

Modality indicates the minimum number of times an instance in one entity can be associated with an instance in the related entity. Modality can have the values of zero or one, two or three are not allowed. The modality symbol is located next to the cardinality symbol, on the inside, i.e., NOT next to the entity box. A modality of one is denoted by a straight vertical line and a modality of zero is denoted by a circle. Like cardinality, modality is indicated at both ends of the relationship.





Reading Modality and Cardinality



Types of Attributes

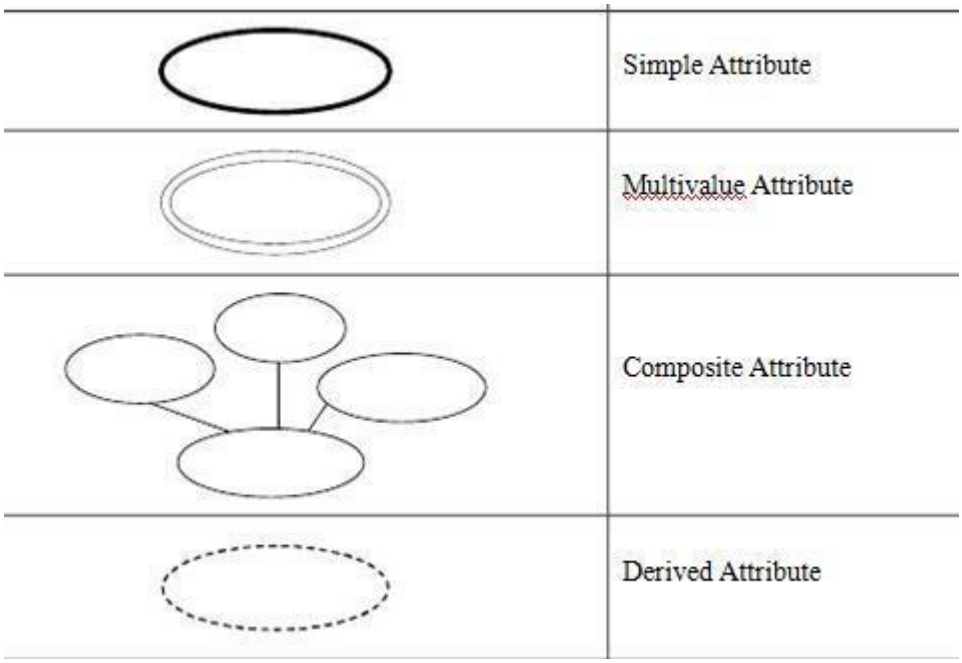
- Simple attribute − Simple attributes are atomic values, which cannot be divided further. For example, a student's phone number is an atomic value of 10 digits.

- Composite attribute − Composite attributes are made of more than one simple attribute. For example, a student's complete name may have first_name and last_name.

- Derived attribute − Derived attributes are the attributes that do not exist in the physical database, but their values are derived from other attributes present in the database. For example, average_salary in a department should not be saved directly in the database, instead it can be derived. For another example, age can be derived from data_of_birth.

- Single-value attribute − Single-value attributes contain single value. For example − Social_Security_Number.

- Multi-value attribute − Multi-value attributes may contain more than one values. For example, a person can have more than one phone number, email_address, etc.



Types of Entity –

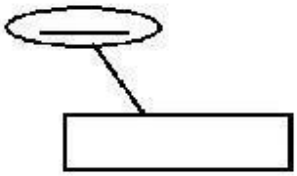
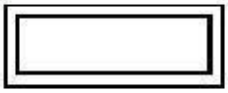
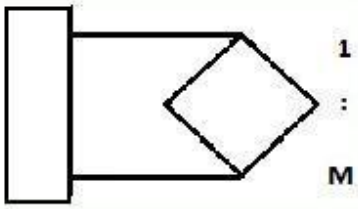
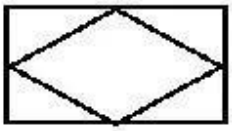
Strong Entity Types

Recursive Entity Types

Weak Entity Types

Composite Entity Types or Associative Entity Types

Notations Of different Entity Type in ER Diagram

	Entity
	Strong Entity Type
	Weak Entity Type
	Recursive Entity Type
	Composite Entity Type (or) Associative Entity

Relationship

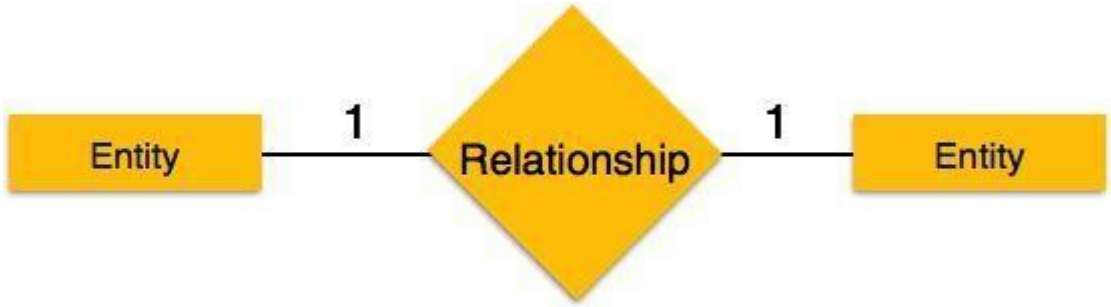
Relationships are represented by diamond-shaped box. Name of the relationship is written inside the diamond-box. All the entities (rectangles) participating in a relationship, are connected to it by a line.

Binary Relationship and Cardinality

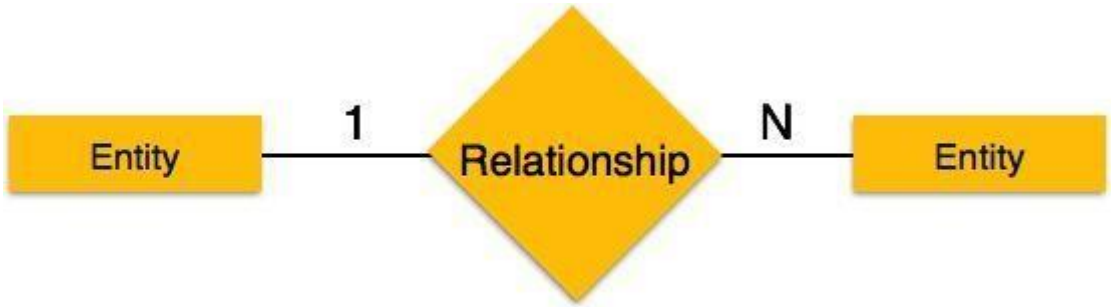
A relationship where two entities are participating is called a binary relationship. Cardinality is the number of instance of an entity from a relation that can be associated with the relation.

- One-to-one − When only one instance of an entity is associated with the relationship, it is marked as '1:1'. The following image reflects that only one

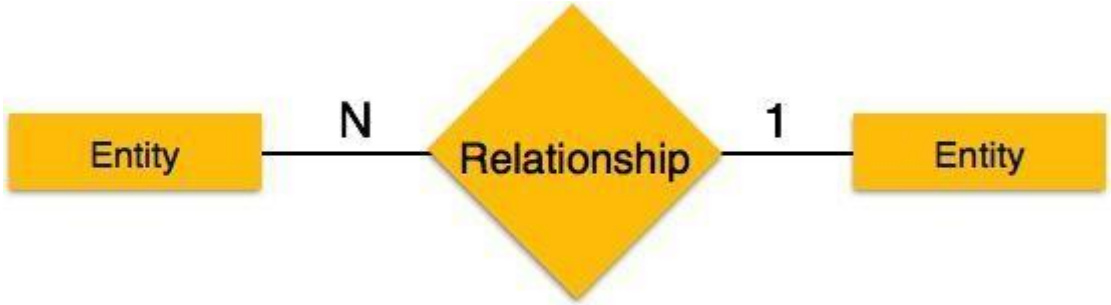
instance of each entity should be associated with the relationship. It depicts one to-one relationship.



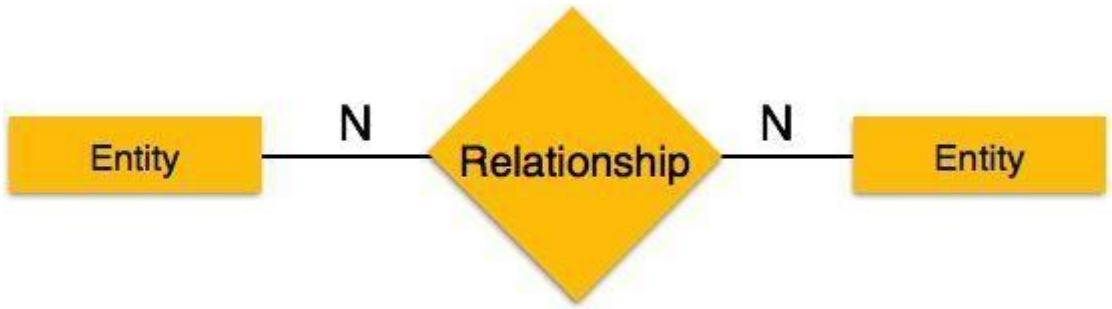
- One-to-many − When more than one instance of an entity is associated with a relationship, it is marked as '1:N'. The following image reflects that only one instance of entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts one-to-many relationship.



- Many-to-one − When more than one instance of entity is associated with the relationship, it is marked as 'N:1'. The following image reflects that more than one instance of an entity on the left and only one instance of an entity on the right can be associated with the relationship. It depicts many-to-one relationship.



- Many-to-many − The following image reflects that more than one instance of an entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts many-to-many relationship.



Basic Concepts of OOP

Object-Oriented Programming (OOP) is a type of programming added to php5 that makes building complex, modular and reusable web applications that much easier.

Class : This is a programmer-defined data type, which includes local functions as well as local data. You can think of a class as a template for making many instances of the same kind (or class) of object.

Object An individual instance of the data structure defined by a class. You define a class once and then make many objects that belong to it. Objects are also known as instance.

Difference Between a Class and an Object

Classes and objects are separate but related concepts. Every object belongs to a class and every class contains one or more related objects.

So what exactly are classes and objects and what is the difference between them?

A Class is static. All of the attributes of a class are fixed before, during, and after the execution of a program. The attributes of a class don't change.

The class to which an object belongs is also (usually) static. If a particular object belongs to a certain class at the time that it is created then it almost certainly will still belong to that class right up until the time that it is destroyed.

An Object on the other hand has a limited lifespan. Objects are created and eventually destroyed. Also during that lifetime, the attributes of the object may undergo significant change.

Class:

A class is the core of any modern Object Oriented Programming language Class is a blueprint of an object that contains variables for storing data and functions to performing

operations on these data.

Object:

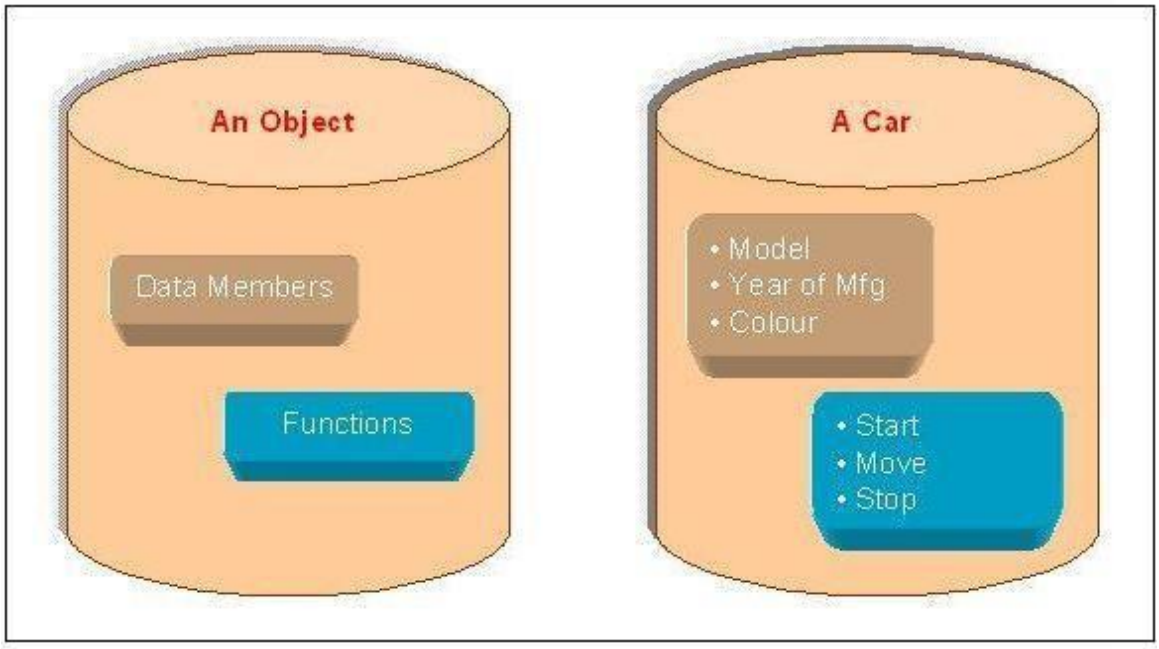
Objects are the basic run-time entities in an object oriented system.They may represent a person,a place or any item that the program has to handle.

"Object is a Software bundle of related variable and methods. "
“Object is an instance of a class” what
are data member and functions

Instance of Object

An instance is an object in memory. Basically you create object and instantiate them when you are using them.

Instance: instance means just creating a reference (copy)



Abstraction, Encapsulation, Inheritance, Polymorphism

Abstraction is the “process of representing only essential features”. That means Abstraction doesn’t show the complexity behind features. Abstraction is used for “Making things more general, simpler, and abstract”.

Real Time Example for Abstraction:

The Best real time example for abstraction is ATM (Automated Teller Machine). We doesnt know how ATM internally works when we are using ATM, we know only select options like withdraw, Balance Inquiry, Mini Statement etc.

Here abstraction hides all unnecessary things, it shows only necessary things.

Encapsulation is defined as "Wrapping up data member and method together into a single unit".

Real time example of Encapsulation is Car Driving, car driver knows how to start car by pressing start button. The driver doesn't know what happened inside when pressing start button. Here the starting process is hidden from driver. So this process can be called as "starting process is encapsulated from driver"

Polymorphism name itself tells "many forms". That is Poly means "Many" morphism means "forms". So Polymorphism meaning in Oop "one name. Or Polymorphism can also define as "same operation may behave differently on different classes".

Real Time Example for Polymorphism

Real time example for polymorphism is "Door", why mean we can use these doors for home, car, lift etc.

Inheritance is defined as "one class (child class) inherits or acquire the property (members) of another class" is "parent child relationship"

Real time example for Inheritance Because child receives all the property from parent.

Parent class : A class that is inherited from by another class. This is also called a base class or super class.

Child Class: A class that inherits from another class. This is also called a subclass or derived class

Association, Aggregation, and Composition

Association is a relationship where all objects have their own lifecycle and there is no owner.

Let’s take an example of Teacher and Student. Multiple students can associate with single teacher and single student can associate with multiple teachers, but there is no ownership between the objects and both have their own lifecycle. Both can be created and deleted independently.

Aggregation is a specialized form of Association where all objects have their own lifecycle, but there is ownership and child objects can not belong to another parent object.

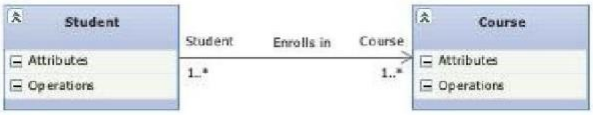
Let’s take an example of Department and teacher. A single teacher can not belong to multiple departments, but if we delete the department, the teacher object will not be destroyed. We can think about it as a “has-a” relationship.

Composition is again specialized form of Aggregation and we can call this as a “death” relationship. It is a strong type of Aggregation. Child object does not have its lifecycle and if parent object is deleted, all child objects will also be deleted.

Let’s take again an example of relationship between House and Rooms. House can contain multiple rooms - there is no independent life of room and any room can not belong to two different houses. If we delete the house - room will automatically be deleted.

Let’s take another example relationship between Questions and Options. Single questions can have multiple options and option can not belong to multiple questions. If we delete the questions, options will automatically be deleted.

Association, Aggregation and Composition



Association – Loose form of relationship (Student can enroll in multiple Course, and A Course can have multiple Student)





Aggregation - Whole part relationship. Part can exist without Whole.
(Engine can exist even if Car is destroyed, the same Engine could be used in a different Car)



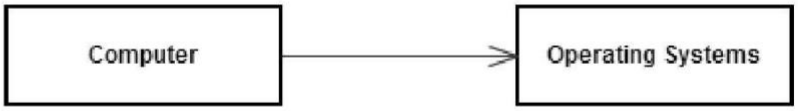


Composition – Stronger form of whole part relationship. Part can not exist without Whole.
(OrderDetail can not exist if Order is deleted. If Order is deleted, OrderDetail also gets deleted)



Computer uses an operating system

Dependency



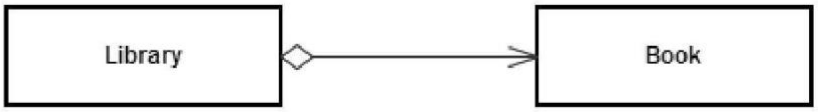
Book "has a" Page

Association



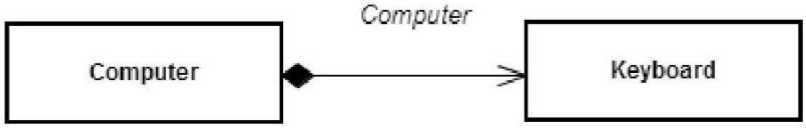
Library "owns a" Book

Aggregration



Keyboard "is a part of" Computer

Composition



Member Variable: These are the variables defined inside a class. This data will be invisible to the outside of the class and can be accessed via member functions. These variables are called attribute of the object once an object is created.

Member function: These are the function defined inside a class and are used to access object data.

Overloading: a type of polymorphism in which some or all of operators have different implementations depending on the types of their arguments. Similarly functions can also be overloaded with different implementation.

Overriding means to override the functionality of an existing method.

Constructor : refers to a special type of function which will be called automatically whenever there is an object formation from a class.

Destructor : refers to a special type of function which will be called automatically whenever an object is deleted or goes out of scope.